

AIM:- Proof of work (Pow) consensus algorithm.

```
import hashlib
import time
```

```
class Block:
```

```
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()
```

```
    def calculate_hash(self):
        block_string =
f"{self.index} {self.timestamp} {self.data} {self.previous_hash} {self.nonce}"
        return hashlib.sha256(block_string.encode()).hexdigest()
```

```
    def mine_block(self, difficulty):
        """Increments nonce until the hash starts with the required number of zeros."""
        target = "0" * difficulty
        print(f"Mining block {self.index}...")

        while self.hash[:difficulty] != target:
            self.nonce += 1
            self.hash = self.calculate_hash()

        print(f"Block mined! Nonce: {self.nonce} | Hash: {self.hash}\n")
```

```
class Blockchain:
```

```
    def __init__(self, difficulty):
        self.difficulty = difficulty
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        """Generates the first block of the blockchain."""
        return Block(0, time.time(), "Genesis Block", "0")

    def get_latest_block(self):
        """Returns the most recent block in the chain."""
        return self.chain[-1]

    def add_block(self, data):
        """Mines a new block and adds it to the chain."""
        latest_block = self.get_latest_block()
```

```
new_block = Block(
    index=latest_block.index + 1,
    timestamp=time.time(),
    data=data,
    previous_hash=latest_block.hash,
)

new_block.mine_block(self.difficulty)
self.chain.append(new_block)

# --- Demonstration ---
if __name__ == "__main__":
    # Difficulty 4 means hashes must start with "0000"
    DIFFICULTY = 4

    my_blockchain = Blockchain(difficulty=DIFFICULTY)

    my_blockchain.add_block("Transaction: Alice sends 5 BTC to Bob")
    my_blockchain.add_block("Transaction: Bob sends 2.5 BTC to Charlie")
```

OUTPUT:-

```
D:\>python Blockchain.py
Mining block 1...
Block mined! Nonce: 42538 | Hash: 0000e8dc869cb23ac649d0d9f72a158b22c3b1a458b1062b44b47667fbe6049e

Mining block 2...
Block mined! Nonce: 52072 | Hash: 0000f8aa9507c76911fdff1a42c37a99786584f0ebd194246fb64e9423edde8a
```

Aim :Dynamic input data

```
import crypto from 'crypto';
import readline from 'readline';

function sha256(data){
return crypto.createHash('sha256').update(data).digest('hex');
}

function buildMerkleTree(dataArray){
if (!dataArray || dataArray.length === 0){
return null;
}
let currentLevel = dataArray.map(item => sha256(item));
console.log('Leaf Hashes: ',currentLevel);

while (currentLevel.length > 1){
const nextLevel = [];
for(let i = 0; i< currentLevel.length; i +=2){
const left = currentLevel[i];
const right = currentLevel[i + 1]? currentLevel[i + 1]: left;
const parentHash = sha256(left + right);
nextLevel.push(parentHash);
}
currentLevel = nextLevel;
console.log('Next Level Hashes: ',currentLevel);
}
return currentLevel[0];
}

const rl = readline.createInterface({
input: process.stdin,
output: process.stdout
});

rl.question('Enter transactions supported by commas (e.g: Txt1,Txt2,Txt3):',(input) => {
//Split the input string trim whitespace, and filter out empty entries
const transactions = input.split(',').map(tx => tx.trim()).filter(tx => tx.length > 0);
if (transactions.length === 0){
console.log('No valid transactions entered.');
```

```
}else{
console.log('Original Data: ',transactions);
const merkleRoot = buildMerkleTree(transactions);
console.log('Final Merkle Root: ',merkleRoot);
}
rl.close();
});
```

OUTPUT:-

```
D:\BCT\TYCS18>node readline.js
Enter transactions supported by commas (e.g: Txt1,Txt2,Txt3):'Hello World'
Original Data: [ "'Hello World'" ]
Leaf Hashes: [ 'df5a9122ad2f3a4d367b146b4ad21a9e9d369b12fa8f9c3efc083c0020c3af3f' ]
Final Merkle Root: df5a9122ad2f3a4d367b146b4ad21a9e9d369b12fa8f9c3efc083c0020c3af3f
```